Document #: P4139R1
Date: 2026-03-23
Project: ISO SC22/WG21 Programming Language C++
Title: Better better lookup
Reply-to: ncm@cantrip.org
Authors: Nathan Myers, Pablo Halpern
Target: C++29
Audience: LEWG

Better better lookup

P3091 proposed a member `get` for associative containers, a monadic analog of `op[]` and `at()`.

While of obvious utility, it is the first instance in the library of a `get()` that can fail, and that takes a runtime-variable key. It is also the first `get()` that (commonly) invokes a loop; every other instance provides access to an immediately available object at a fixed compile-time offset. When considered in Sofia, the vote (weakly) favored retaining the name `get`, but discussion since has indicated receptivity to an alternative. This paper is about choosing an alternative.

Discussion

P3091 offered `get_optional`, `lookup`, and `lookup_optional`, which failed to evoke enthusiasm. Interest at the time was focused on semantics, with discussion of names felt by many to be a distraction. The only criterion for a good name mentioned was "short", which seems to the authors secondary to descriptive clarity.

The only alternative seriously considered at the time was `lookup`, which was rejected by weak consensus. The goal of this paper is not to advocate for a particular name, but rather for a practical choice consistent with other usage in the library, and more descriptive and apt than the opaque `get`.

As a reminder, in use, the member would typically appear in a context like

```
  if (auto maybe = container.xxxxx(key)) {
    use(*maybe);
  }
```

or

```
  use(container.xxxxx(key).value_or(mapped{}));
```

The library has accumulated an assortment of `try_xxxxx` members that share the feature of reporting failure, but return a variety of result types, some presenting an iterator adjacent to where an element *would have* been inserted or erased, or `end()`; sometimes in a pair with a boolean flag.

Decades before those, containers got `at(key)` that throws on a miss, and `operator[](key)` that on a miss returns a reference to a default-initialized element, inserted; but also might throw if insertion fails. That these return an element value reference makes them closely akin to the feature proposed in P3091.

Proposal

Suggested names include `try_at(key)`, `lookup(key)`, and `try_lookup(key)`, but suggestions for other alternatives consistent with existing library usage are welcome.

The feature test macro should be made to match the new chosen name.

The same member should be added to `vector`, `inplace_vector`, and other containers that expose a member `at()`.

It should be no-throw if operations used (comparison, hash, etc) are.

History

R1: Add Pablo Halpern as co-author; note feature-test macro to be changed; identify correct (C++29) target; add to other indexable containers; suggest no-throw semantics.

Reference

[1]: [P3091R3] Pablo Halpern - Better lookups for `map` and `unordered map` https://wg21.link/p3091r3